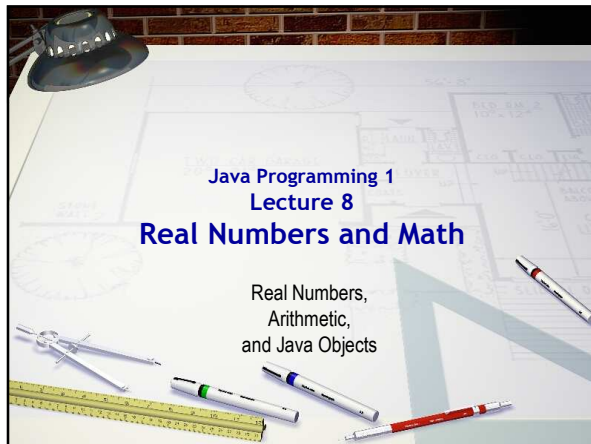
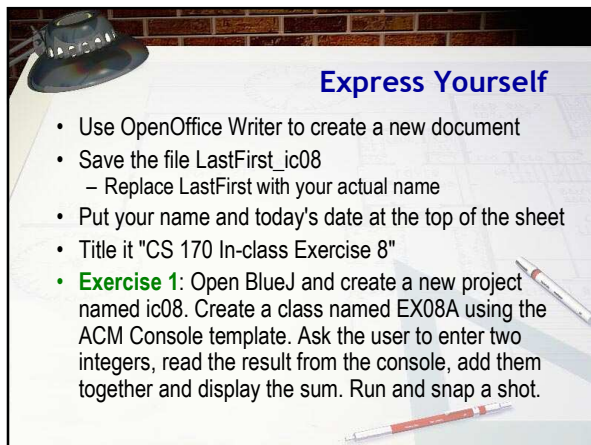
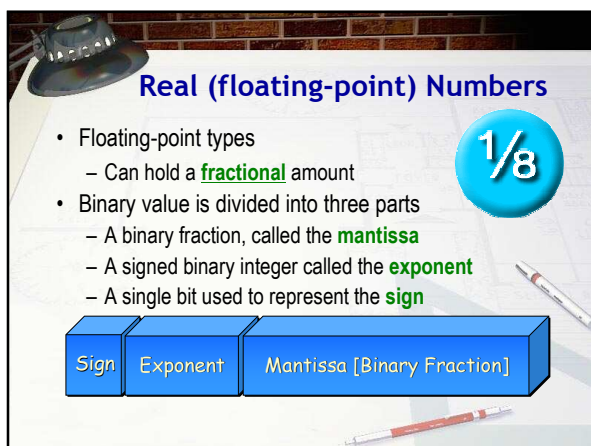
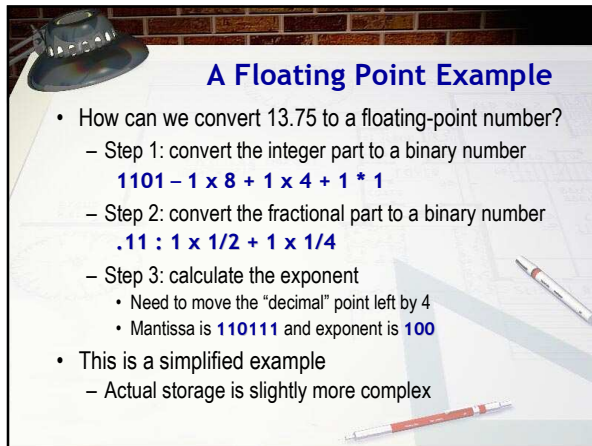




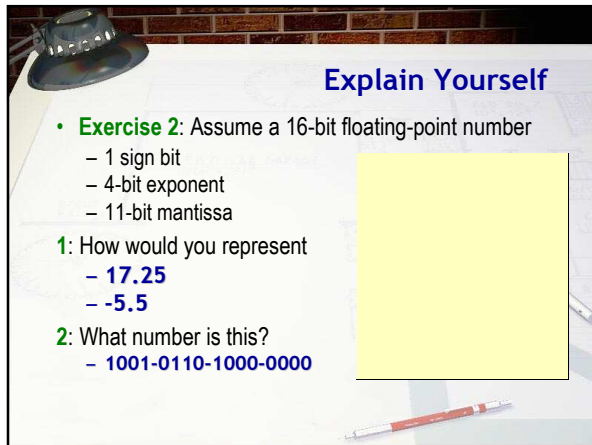






A Floating Point Example

- How can we convert 13.75 to a floating-point number?
 - Step 1: convert the integer part to a binary number
 $1101 - 1 \times 8 + 1 \times 4 + 1 \times 1$
 - Step 2: convert the fractional part to a binary number
 $.11 : 1 \times 1/2 + 1 \times 1/4$
 - Step 3: calculate the exponent
 - Need to move the "decimal" point left by 4
 - Mantissa is **110111** and exponent is **100**
- This is a simplified example
 - Actual storage is slightly more complex



Explain Yourself

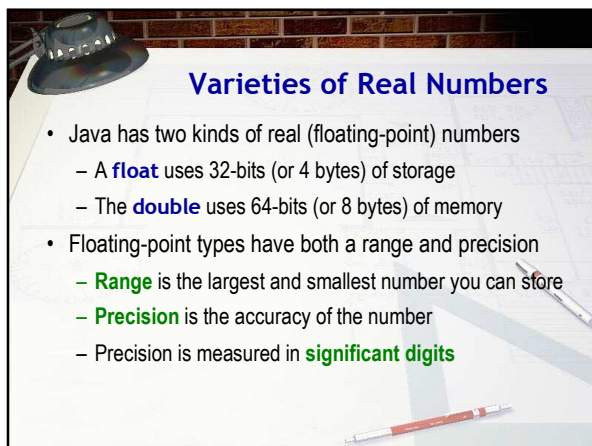
- Exercise 2:** Assume a 16-bit floating-point number
 - 1 sign bit
 - 4-bit exponent
 - 11-bit mantissa

1: How would you represent

- 17.25**
- 5.5**

2: What number is this?

- 1001-0110-1000-0000**



Varieties of Real Numbers

- Java has two kinds of real (floating-point) numbers
 - A **float** uses 32-bits (or 4 bytes) of storage
 - The **double** uses 64-bits (or 8 bytes) of memory
- Floating-point types have both a range and precision
 - Range** is the largest and smallest number you can store
 - Precision** is the accuracy of the number
 - Precision is measured in **significant digits**

Range and Precision

- [illegible]

Defining Real Variables

- Use **double** or **float** for type name
- Floating-point literals can be written in two ways
 - Decimal notation: **3.45, .7653, 2.0**
 - Exponential notation: **1.0E6, 2.7563e-15**

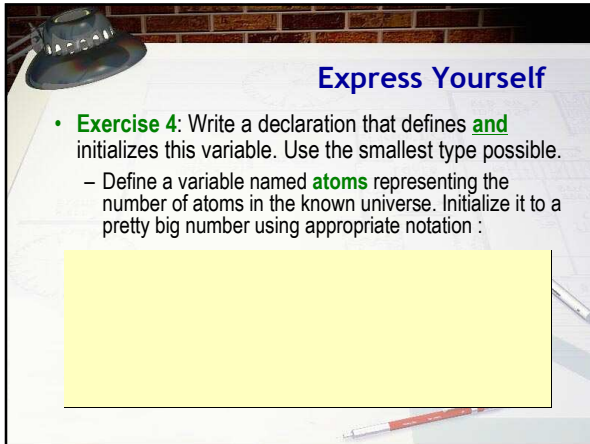
```
double first    = .234;  
double second  = 7.5432;  
double third   = 1.234e-12;  
float fourth   = 3.2F;
```



- **Must** use a suffix of **F** or **f** when writing a float literal
- **May** use a suffix of **D** or **d** when writing a double literal

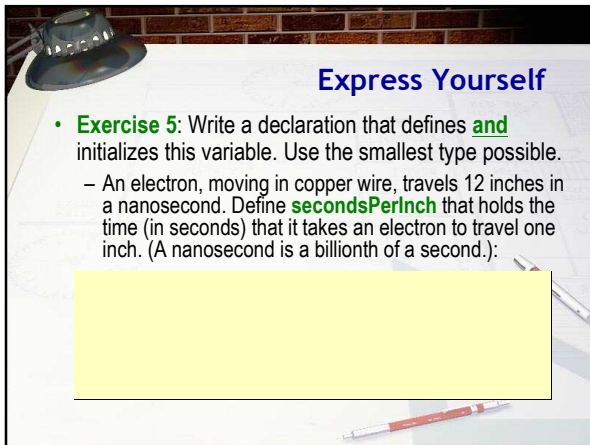
Express Yourself

- **Exercise 3:** Write a declaration that defines **and** initializes this variable. Use the smallest type possible.
 - Define a variable named **kids** that holds the average number of children per American family. Initialize it to one and three-quarters using decimal notation:



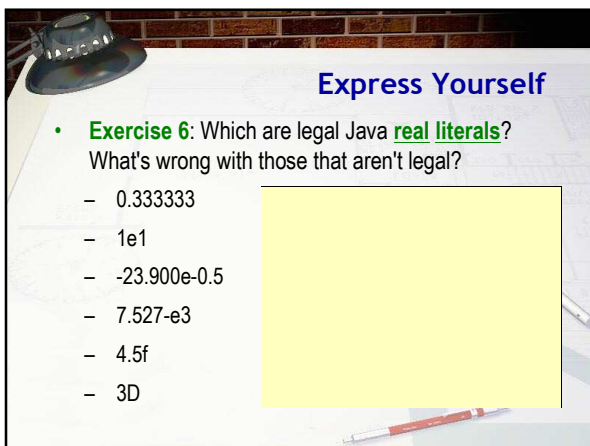
Express Yourself

- **Exercise 4:** Write a declaration that defines and initializes this variable. Use the smallest type possible.
 - Define a variable named **atoms** representing the number of atoms in the known universe. Initialize it to a pretty big number using appropriate notation :



Express Yourself

- **Exercise 5:** Write a declaration that defines and initializes this variable. Use the smallest type possible.
 - An electron, moving in copper wire, travels 12 inches in a nanosecond. Define **secondsPerInch** that holds the time (in seconds) that it takes an electron to travel one inch. (A nanosecond is a billionth of a second.):



Express Yourself

- **Exercise 6:** Which are legal Java real literals? What's wrong with those that aren't legal?
 - 0.333333
 - 1e1
 - -23.900e-0.5
 - 7.527-e3
 - 4.5f
 - 3D

Floating-point Input

- Use the same "pattern" as you did for integer input
 - Only the method names are different
- With **Scanner**, use **nextFloat()** or **nextDouble()**

```
24 System.out.print ("Enter the gallons of fuel used: ");
25 gallons = scan.nextDouble();
26
27
```

- With ACM, use **readFloat()** or **readDouble()**

```
22 double gallons = readDouble("Enter gallons of gas (3.5) : ");
23
24
```

Express Yourself

- **Exercise 7:** Let's modify the program we started out with (EX08A). Create a double or float variable named price. Ask the user to enter the price of a hamburger, and store the result in your variable. Now, create a second int variable called pennies. Multiply the price by 100, and store the result in pennies. Print out the number of pennies.
- What happens when you compile?

Conversion and Assignment

- Java allows you to write assignments like this:

```
double a = 5;
```

- This is called a **widening conversion**
 - Assignment between types with no lost information

```
double d = 123.45F; // OK
float f = 123.45;   // Not OK
int x = 123;
byte b = x;          // Not OK
long l = x;           // OK
```

Casting

- Sometimes you know that no data will be lost
 - Or, the data loss may be acceptable
- You can tell Java to “try” to store closest value
 - This is called a “**cast**” or “**type cast**”
 - It creates a temporary expression value
- Cast syntax : **(new-type) value**

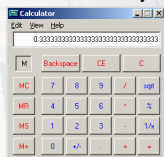
```
int x    = 123  
byte b   = (byte) x;           // OK  
byte b   = (byte) 256;         // OK???  
int n    = (int) .999;         // OK???
```

Express Yourself

- **Exercise 8:** Use casting to fix the compiler error when you store the double value into pennies. Run your program and enter a value of 4.35 for the hamburger. How many pennies did you collect? Shoot me a screenshot.

Floating Point Limitations

- Floating-point numbers represent approximations
 - They are especially **unsuited** for monetary applications
 - The value .1 cannot be exactly represented in binary
- Similar to the problem with $1/3$ in decimal
- If you enter this into a calculator you see something that looks like:
- Called a **"representational error"**
 - Your book calls it a rounding error



What About Money?

- What about money, if floating-point isn't that good?
 - We can get closer, by using `Math.round()`
- **Exercise 9:** Try it, re-run and shoot a pic.
- Better, we could create an accurate `Money` class
 - Use `longs` to represent mills accurately
- Java already has a similar class called `BigDecimal`

```
BigDecimal d = new BigDecimal("4.35");  
BigDecimal e = new BigDecimal("100");  
BigDecimal f = d.multiply(e);  
System.out.println(f);
```

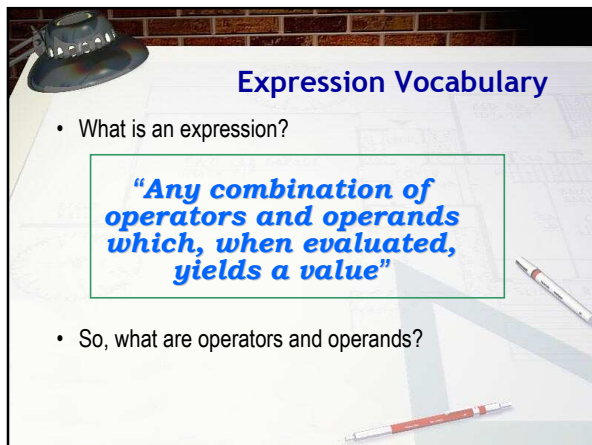
Express Yourself

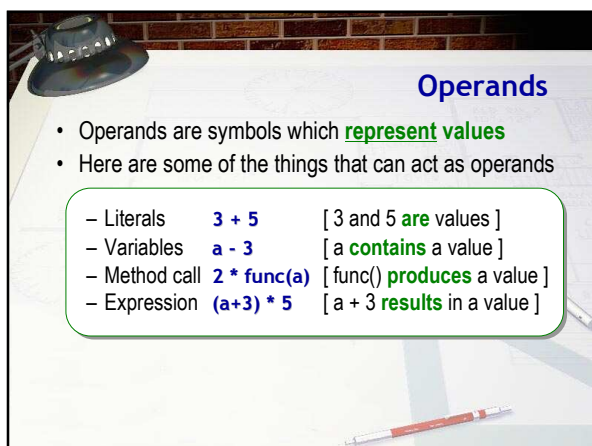
- **Exercise 10:** Fix your program again, this time to use `BigDecimal` for your price variable. That means you'll have to read String input to construct your variable. Run and shoot me a picture of the result.

Guidelines: What To Use

- Choosing between decimal and scientific notation
 - Very large or very small numbers, use scientific
- Choosing between `float` and `double`
 - `double` is natural size for most modern hardware
 - Absent a compelling reason to use `float`, use `double`
- Choosing between `double` and `int`
 - Using `double` is significantly slower and less accurate
 - Use `int` unless you need fractions, then use `double`
- Money: use `BigDecimal` or `longs` for mills







Operators

- Operators are symbols that **perform an operation**
 - Operation may be performed on one or more operands
- Every operator has these three characteristics:
 - Arity**: number of operands required
 - Unary, binary, tertiary (ternary)
 - Precedence**: order in which operations are performed
 - Associativity**: determines whether operations should occur left-right or right-left

Evaluation

- Key concept**: every expression **produces a value**
 - Value may be used anywhere a literal value is used

```
a = 7;  
a = 3 + 4;
```

- Value produced may be stored, or used immediately

```
int fontsize = 72 / 2; // 36 points  
f1 = new Font("Serif", Font.BOLD, fontsize);  
f2 = new Font("Serif", Font.BOLD, 72/2);
```

Numeric Operators

- These are the five commonly used numeric operators

+	Addition
-	Subtraction or unary minus
*	Multiplication
/	Division (both integer and real)
%	Remainder or modulus (also works with reals)

```
int a = 7, b = 12;  
int diff      = b - a;  
int product   = b * a;  
int quotient  = b / a;  
int remainder = b % a;
```

Integer Division Operations

- Integer division works like primary-school long division
 - The quotient is calculated, remainder discarded
 - The result, an integer, is truncated, not rounded
- The remainder operator returns the "remainder" part
 - 12 % 7 is 5: 7 "goes into" 12 once, with 5 left over
 - The sign of the result is the sign of the numerator
- Exercise 11:** Here are some challenge problems

```
int a = 5 % 3;  
int b = 0 % 2;  
int c = 12 % 12;  
int d = 2 % 0;
```

Precedence

- Precedence is a rule used to evaluate an expression
 - Precedence determines "which operator goes first"
 - Operators with "higher" precedence go first

```
int x = 5 - 5 * 3; // -10, not 0
```

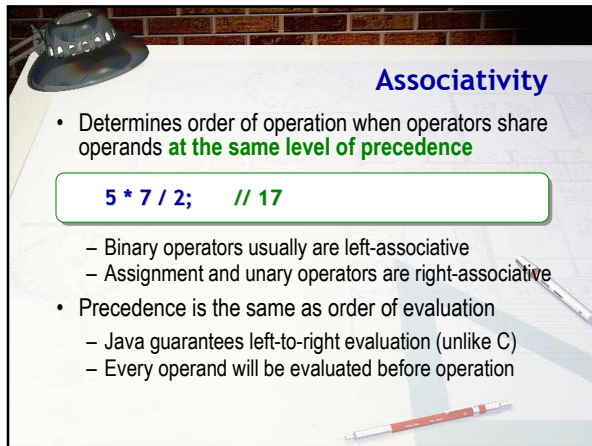
- Can be overridden using parentheses

```
int x = (5 - 5) * 3; // 0
```

- See complete chart in Appendix E in your book

Precedence Levels

↑ Higher	unary postfix operators	++ --
	unary prefix operators	++ -- + -
	multiplicative operators	* / %
	additive operators	+ -
↓ Lower	assignment operators	=

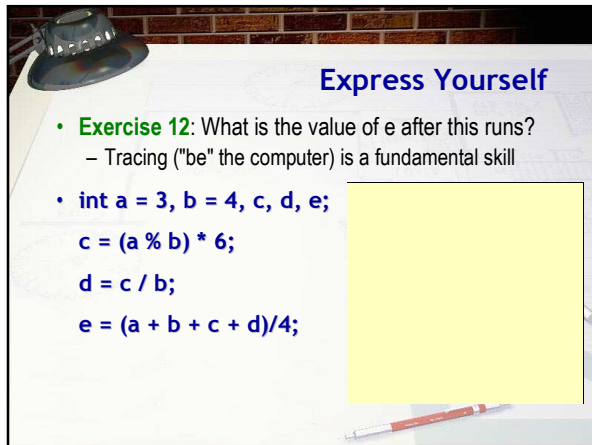


Associativity

- Determines order of operation when operators share operands **at the same level of precedence**

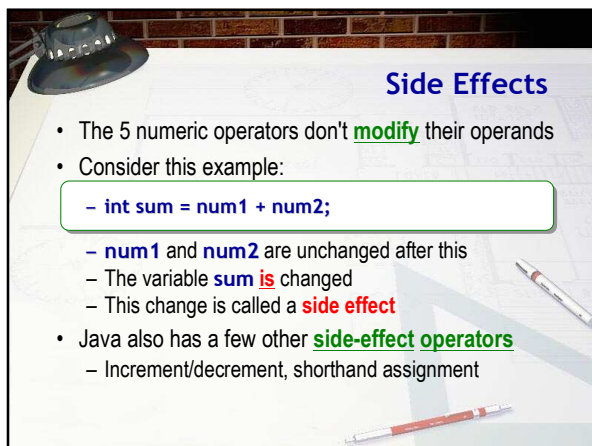
`5 * 7 / 2;    // 17`

- Binary operators usually are left-associative
- Assignment and unary operators are right-associative
- Precedence is the same as order of evaluation
 - Java guarantees left-to-right evaluation (unlike C)
 - Every operand will be evaluated before operation



Express Yourself

- Exercise 12:** What is the value of e after this runs?
 - Tracing ("be" the computer) is a fundamental skill
- `int a = 3, b = 4, c, d, e;`
`c = (a % b) * 6;`
`d = c / b;`
`e = (a + b + c + d)/4;`



Side Effects

- The 5 numeric operators don't **modify** their operands
- Consider this example:

`int sum = num1 + num2;`

 - `num1` and `num2` are unchanged after this
 - The variable `sum` **is** changed
 - This change is called a **side effect**
- Java also has a few other **side-effect operators**
 - Increment/decrement, shorthand assignment

Shorthand Assignment

- To sum an amount in `runningTotal` you'd write:

```
runningTotal = runningTotal + currentAmount;
```

- You have to write the word `runningTotal` twice

- Java has a **shorthand** way to write the same thing:

```
runningTotal += currentAmount;
```

More Shorthand Assignment

- There are shorthand versions of all binary operators:

```
a += b; // same as a = a + b;  
a -= b; // same as a = a - b;  
a /= b; // same as a = a / b;  
a *= b; // same as a = a * b;  
a %= b; // same as a = a % b;
```

Express Yourself

- Exercise 13:** Write equivalent "shorthand" if possible:

```
x = 2 * x;  
x = x + y - 2;  
sum = sum + num;  
z = z * x + 2 * z;  
y = y / (x + 5);
```

Up and Down

- Increment and decrement operators
 - Unary operators (++ , --) that add or subtract one
 - Can **only** be applied to a variable
 - Produce both a value and a side-effect
- You may use the operator before or after the variable
 - Both have **exactly the same side-effect** on variable
 - Variable is **always** incremented or decremented by 1

```
int a = 5;  
a++;      // a is now 6  
++a;      // a is now 7
```

Pre and Post

- The "expression" or "returned" value is **different**
- With **pre-increment** (before the variable)
 - The variable is updated
 - The new value is returned as the expression's value
- With **post-increment** (after the variable)
 - The original value is saved temporarily
 - The variable is updated
 - The original value is returned as the expression's value

```
int first = 1, second = 2;  
int third = first++;    // third==1, first==2  
int fourth = ++second; // fourth, second == 3
```

Express Yourself

- Exercise 14:** What value does each variable contain after each statement is executed? (U if not defined)
- ```
int a = 5, b = 6, c;
a = b++ + 3;
c = 2 * a + ++b;
b = 2 * ++c - a++;
```

---

---

---

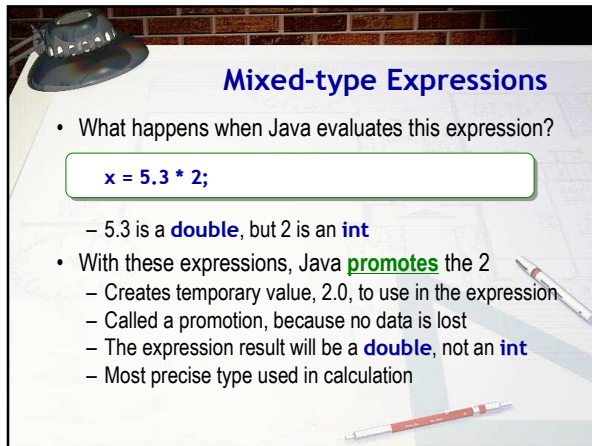
---

---

---

---

---



### Mixed-type Expressions

- What happens when Java evaluates this expression?  
`x = 5.3 * 2;`
  - 5.3 is a **double**, but 2 is an **int**
- With these expressions, Java **promotes** the 2
  - Creates temporary value, 2.0, to use in the expression
  - Called a promotion, because no data is lost
  - The expression result will be a **double**, not an **int**
  - Most precise type used in calculation

---

---

---

---

---

---

---



### Promotion Rules

- Java data is arranged in a hierarchy
- Types holding more info at the top, less at bottom
- Hierarchy used to determine type of promotion
  - 1) Expression contains **byte**, **short** or **int**
  - 2) Expression contains **long**
  - 3) Expression contains floating-point type
  - 4) Expression contains **float** and **double**
- Promotion occurs only when encountered  
`double value = 5 / 4 / 3.0`

---

---

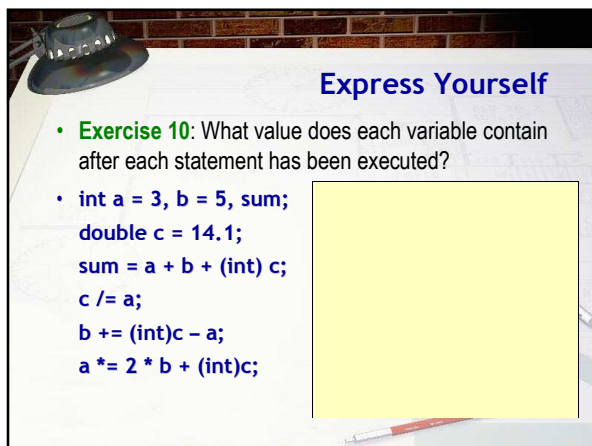
---

---

---

---

---



### Express Yourself

- Exercise 10:** What value does each variable contain after each statement has been executed?
- `int a = 3, b = 5, sum;`  
`double c = 14.1;`  
`sum = a + b + (int) c;`  
`c /= a;`  
`b += (int)c - a;`  
`a *= 2 * b + (int)c;`

---

---

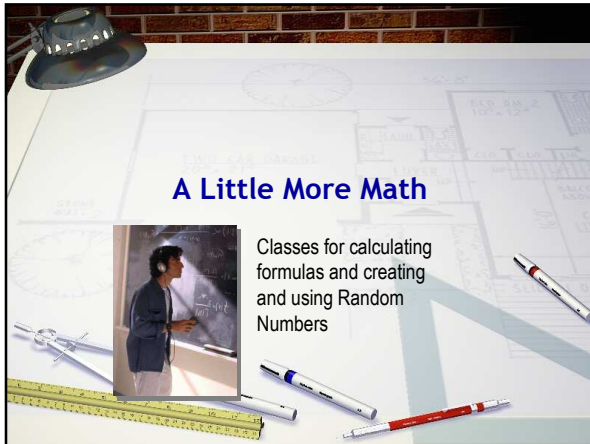
---

---

---

---

---



### A Little More Math

Classes for calculating formulas and creating and using Random Numbers

---

---

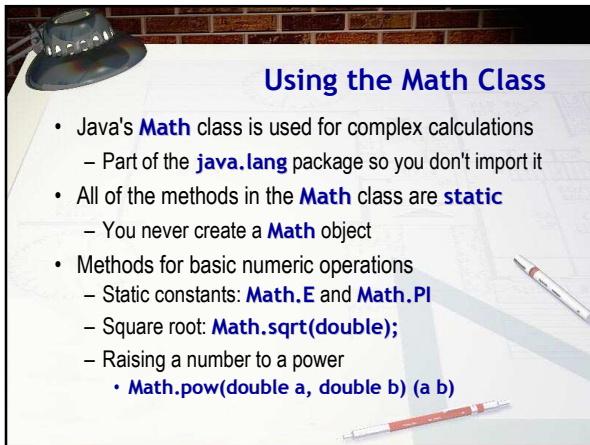
---

---

---

---

---



### Using the Math Class

- Java's **Math** class is used for complex calculations
  - Part of the **java.lang** package so you don't import it
- All of the methods in the **Math** class are **static**
  - You never create a **Math** object
- Methods for basic numeric operations
  - Static constants: **Math.E** and **Math.PI**
  - Square root: **Math.sqrt(double);**
  - Raising a number to a power
    - **Math.pow(double a, double b) (a b)**

---

---

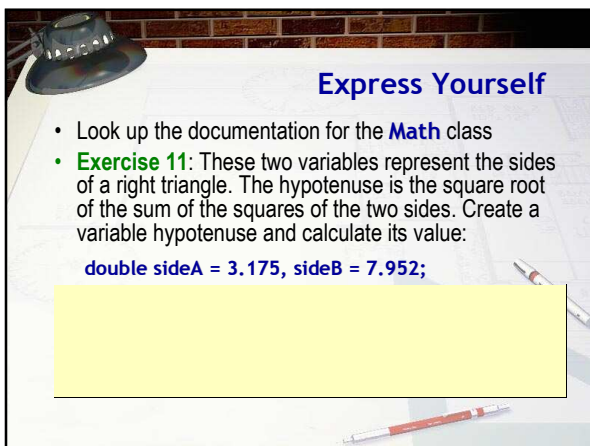
---

---

---

---

---



### Express Yourself

- Look up the documentation for the **Math** class
- **Exercise 11:** These two variables represent the sides of a right triangle. The hypotenuse is the square root of the sum of the squares of the two sides. Create a variable hypotenuse and calculate its value:  
**double sideA = 3.175, sideB = 7.952;**

---

---

---

---

---

---

---

### Express Yourself

- Look up the documentation for the **Math** class
- Exercise 12:** The variable **total** may be positive or negative. Use a method or methods in the **Math** class to compute and print the absolute value of **total**.  
`double total = scan.nextDouble();`



---

---

---

---

---

---

---

### Random Numbers with Math

- The **Math.random()** method is very easy to use
  - Generates random numbers between 0.0 and 1.0
  - 0.0 is included and 1.0 is excluded
- Normally, we want a wider range and a different type
  - Use multiplication to get the "number of instances"
  - Use casting to convert to the correct type
  - Use addition to put the number in the range you want

```
int die = (int) (Math.random() * 6) + 1;
```

---

---

---

---

---

---

---

### Let 'em Roll

- Imagine a lottery game where winner must guess six numbers between 39 and 173 (inclusive)
  - Here's how you'd generate the first number in the sequence

```
int min = 39;
int max = 173;
int range = max - min + 1;
int p1 = (int) (Math.random() * range) + min;
```



---

---

---

---

---

---

---



### Express Yourself - Wild Ovals

- **Step 1:** Create a new applet in BlueJ called **WildOvals**
  - Paste the starter code from the class links page
- **Step 2:** Complete the program using **Math.random()**
  - In **paint()**, draw a solid, colored oval on the screen
  - Use random numbers for the oval coordinates and size
  - Generate 4 ints falling inside the area of the applet
  - Use the smaller of them as x and y
  - Calculate a value for width and height.
- **Step 3:** Generate a random color for the oval.
- **Exercise 14:** Snap a picture of your applet running

---

---

---

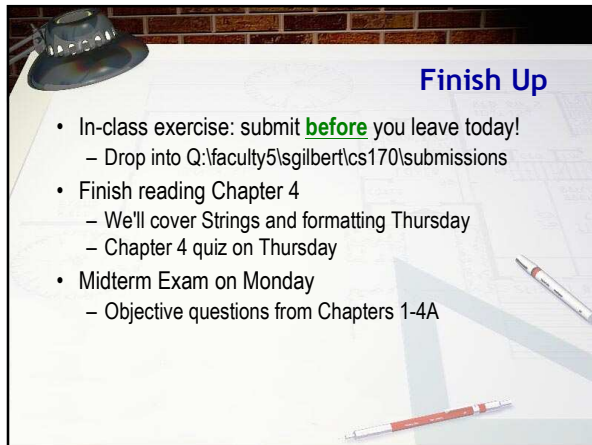
---

---

---

---

---



### Finish Up

- In-class exercise: submit **before** you leave today!
  - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Finish reading Chapter 4
  - We'll cover Strings and formatting Thursday
  - Chapter 4 quiz on Thursday
- Midterm Exam on Monday
  - Objective questions from Chapters 1-4A

---

---

---

---

---

---

---

---